

Session 4: Functions, methods and modules

Naming a piece of work, reusing it, and the objects every Python value is made of.

Data Science and Advanced Programming · UNIL ·
2026

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

A name for an idea

Session 3's script, and its limits

Session 3 valued the ledger in one long script. Now suppose three reports need a position's value, and a fourth needs only the total.

- Copy and paste the same lines four times? Every copy can go wrong on its own.
- Change the rule once, and you must find every copy.

Today: give a piece of work **a name**, write it **once**, and **call** it wherever you need it.

```
$ python portfolio.py
```

```
not run yet
```

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
```

```
not run yet
```

Today: one script, three functions

1. Calling functions: what a call is
2. Writing functions: `def` and `return`
3. Signatures and arguments: positional, keyword, default
4. Scope: whose names are these?
5. Methods: `sum(x)` and `x.sum()`
6. Modules and `import`
7. Classes and objects: numpy, pandas, and our own

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
```

not run yet

Read it cold

Two minutes, on your own.

- Find the three words `def`. What do you think each one does?
- Line 21 uses `total_value`. Where does it come from?
- **Guess the output.** Which lines print?

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price): Three definitions
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"] The data
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20  Using the functions
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

The shape of the script

Lines	Job
2	bring in a module: <code>math</code>
4–15	define three functions
17–19	the data (session 3)
21–25	use the functions

The functions come **first**, but they are only **definitions**. Nothing in them runs until line 21 calls one.

```
$ python portfolio.py
```

```
not run yet
```

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

What it prints

You do not need to run anything now. Bottom left is the output of one run of the script.

Four lines. The first is session 3's total, 3,338.20 . The second is session 3's **15 years**, now computed by a formula instead of a loop.

By the end of today, you can explain every character of it.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), year A call (session 1) te=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

REMINDER

You have been calling functions since session 1

Call	Seen in
<code>print("hello")</code>	session 1
<code>round(share, 2)</code>	session 1 workshop: "keep what comes back"
<code>len(ticker)</code> , <code>type(x)</code> , <code>int(raw)</code>	session 2
<code>range(3)</code> , <code>sorted(prices)</code> , <code>sum(shares)</code>	session 3

Session 1 already said: a **call** is a name, then parentheses, with what you hand over inside. Today we look inside the parentheses, and behind the name.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print("Methods (session 3)", price=102.3, shares=4, sum(shares))
25  print(" ", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

REMINDER

And methods since session 3

```
shares.append(3), line.split(",") ,
price.get("MSFT", 0.0), values.sum() .
```

Session 3 called them **methods**: operations that belong to a type, written **after a dot**.

Part 5 explains why some operations are written `sum(shares)` and others `arr.sum()` .

PART 1

Calling a function

What happens when Python meets a name followed by parentheses.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

One call

```
$ python portfolio.py
```

```
total 3,338.20 3338.2
```

```
15 11
```

```
409.2 16
```

```
aapl, nesn, voo <class 'function'>
```

Anatomy of a call

line 22	<code>round(total, 1)</code>
the function	<code>round</code>
the arguments	<code>total, 1</code>
the parentheses	<code>()</code>

- The **function** is the name before the parentheses.
- The **arguments** are the values handed over, separated by commas.
- The parentheses **call** it: they make it run now.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

A call is replaced by its value

A call is an expression, like `2 + 3`: Python works it out, and **the call is replaced by the value it gives back**.

line 22 `round(total, 1)`

↓ names become values

`round(3338.2, 1)`

↓ the function runs

replaced by `3338.2`

That value can go anywhere a value can: into a variable, into another call, into a `print`.

```
$ python portfolio.py
```

```
total 3,338.20 3338.2 round(total, 1)
```

```
15 11
```

```
409.2 16
```

```
aapl, nesn, voo <class 'function'>
```

portfolio.py

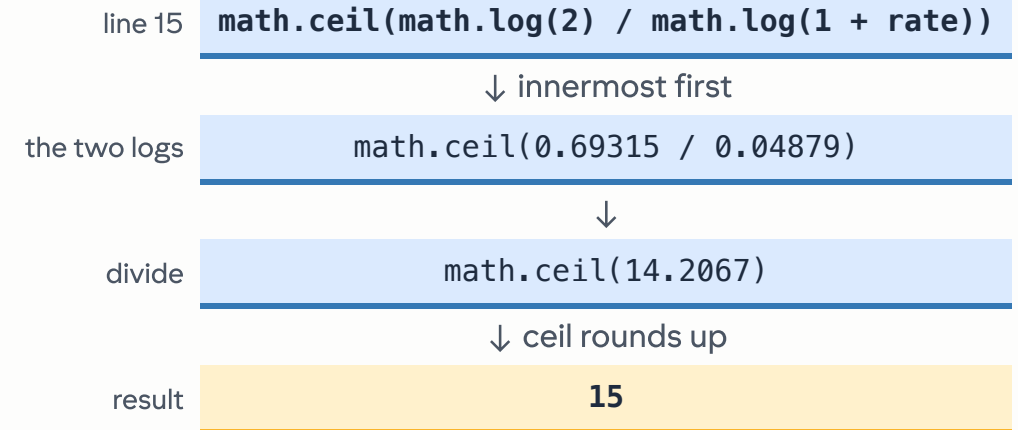
```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17     ticks = ["AAPL", "NESN", "V00"]
18     shares = [10, 4, 2]
19     price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21     total = total_value(ticks, shares, price)
22     print(f"total {total:,.2f}", round(total, 1))
23     print(years_to_double(), years_to_double(rate=0.07))
24     print(position_value(price=102.3, shares=4), sum(shares))
25     print(", ".join(ticks).lower(), type(total_value))
```

Calls inside calls

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

Calls inside calls: read from the inside out

Line 15 has four calls nested together. With `rate = 0.05`:



Session 3 needed a `while` loop for this. Here a formula does it in one line: 15 years again.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

The built-in functions

Python starts with these ready to call, no `import` needed:

Function	Gives back	Function	Gives back
<code>print(x)</code>	None (it shows <code>x</code>)	<code>len(x)</code>	the number of items
<code>sum(x)</code> , <code>min(x)</code> , <code>max(x)</code>	total, smallest, largest	<code>round(x, n)</code> , <code>abs(x)</code>	a number
<code>int(x)</code> , <code>float(x)</code> , <code>str(x)</code>	a converted value	<code>type(x)</code>	the class of <code>x</code>
<code>sorted(x)</code> , <code>range(n)</code>	a new list, a range	<code>help(f)</code>	the documentation of <code>f</code>

There are about 70. You have now met most of the ones you will use.

```

1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 tickers = ["AAPL", "NESN", "V00"]
18 shares = [10, 4, 2]
19 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21 total = total_value < No parentheses: not called
22 print(f"total {total:,.2f}", round(total, 1))
23 print(years_to_double(), years_to_double(rate=0.07))
24 print(position_value(price=102.3, shares=4), sum(shares))
25 print(", ".join(tickers).lower(), type(total_value))

```

```
$ python portfolio.py
```

```
Traceback (most recent call last):
```

```
File "portfolio.py", line 22, in <module>
```

```
    print(f"total {total:,.2f}", round(total, 1))
          ~~~~~^~~~~~
```

```
TypeError: unsupported format string passed to function.__format__
```

Forgot the parentheses

Without `( )`, the function is **not called**. `total` becomes the function itself, not its result.

Line 22 then tries to format a function as money, and crashes. (Output line 4 of the original shows the type of a function: `<class 'function'>`.)

A name is not a call. The parentheses are what make it run.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14     def years_to_double(rate=0.05): can be called with nothing
15         return math.ceil(math.log(2) / math.log(1 + rate))
16
17     tickers = ["AAPL", "NESN", "V00"]
18     shares = [10, 4, 2]
19     price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21     total = total_value(tickers, shares, price)
22     print(empty(): still a call .2f}", round(total, 1))
23     print(years_to_double(), years_to_double(rate=0.07))
24     print(position_value(price=102.3, shares=4), sum(shares))
25     print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11 years_to_double()
409.2 16
aapl, nesn, voo <class 'function'>
```

No arguments? Still write `()`

`years_to_double()` on line 23 hands over **nothing**: its only parameter has a default. The parentheses are still needed, **empty**.

- `years_to_double()` : call it, get 15.
- `years_to_double` : only the name, not a call (the previous slide).

The same holds for methods: `ticker.lower()`, `values.sum()`, `ledger.head()`. `Empty ()` at the end means "run it now".

PART 2

Writing your own functions

Give a piece of work a name, write it once, use it everywhere.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

Why write functions?

- **Name an idea.** `total_value(tickers, shares, price)` says what line 21 does. Session 3's loop made you work it out.
- **Write it once.** Fix a mistake in one place, and every call is fixed.
- **Check it alone.** You can test `position_value` with numbers you know, before trusting the rest.
- **Read at two levels.** Lines 21–25 tell the story; lines 4–15 hold the details.

Nearly every program you read from now on, including AI-written code, is organised into functions.

portfolio.py

```
1 # portfolio.py – the ledger, organised into functions
2 import math
3
4 def position_value(shares, price):
5     """Value of one position, in CHF."""
6     return shares * price
7
8 def total_value(tickers, shares, price):
9     total = 0.0
10    for i in range(len(tickers)):
11        total += position_value(shares[i], price[tickers[i]])
12    return total
13
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 tickers = ["AAPL", "NESN", "V00"]
18 shares = [10, 4, 2]
19 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21 total = total_value(tickers, shares, price)
22 print(f"total {total:,.2f}", round(total, 1))
23 print(years_to_double(), years_to_double(rate=0.07))
24 print(position_value(price=102.3, shares=4), sum(shares))
25 print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

Anatomy of def

line 4	def position_value(shares, price):
define	def
its name	position_value
its parameters	(shares, price)
body follows	:

- The **body** is the indented block (lines 5–6), as for `for` and `if`.
- **Parameters** are names for the values a call will hand over.
- A **docstring** (line 5) says in one sentence what the function does.
- `return` ends the function and sends a value back to the caller.
- **Names:** `snake_case`, like variables.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price): Stored, not run
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20     First real run
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

Defining is not running

Python reads the file top to bottom, as always. At a `def`, it **stores** the function under its name, and **skips** the body.

Line	What happens
2	load the <code>math</code> module
4, 8, 14	store three functions; run nothing inside
17–19	create the data
21	first call: now lines 9–12 run, and line 11 calls line 6 three times

A function that is never called never runs, even if its body is full of mistakes.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13     2. back to line 21
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 tickers = ["AAPL", "NESN", "V00"]
18 shares = [10, 4, 2]
19 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20     1. jump in
21 total = total_value(tickers, shares, price)
22 print(f"total {total:,.2f}", round(total, 1))
23 print(years_to_double(), years_to_double(rate=0.07))
24 print(position_value(price=102.3, shares=4), sum(shares))
25 print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

A call's journey

line 21: total_value(...) → jump in

lines 9–11: total = 0.0, start the loop

line 6: position_value(10, 190.5) → 1905.0

line 6: position_value(4, 102.3) → 409.2

line 6: position_value(2, 512.0) → 1024.0

line 12: return total → 3338.2

line 21: total = 3338.2 ← back where we left

Each call **jumps** into the function, runs its body, and **comes back** with a value.

```

1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      print(shares * price) < Shows the value, returns None
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 tickers = ["AAPL", "NESN", "V00"]
18 shares = [10, 4, 2]
19 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21 total = total_value(tickers, shares, price)
22 print(f"total {total:,.2f}", round(total, 1))
23 print(years_to_double(), years_to_double(rate=0.07))
24 print(position_value(price=102.3, shares=4), sum(shares))
25 print(", ".join(tickers).lower(), type(total_value))

```

```

$ python portfolio.py
1905.0
Traceback (most recent call last):
  File "portfolio.py", line 21, in <module>
    total = total_value(tickers, shares, price)
            ~~~~~
  File "portfolio.py", line 11, in total_value
    total += position_value(shares[i], price[tickers[i]])
TypeError: unsupported operand type(s) for +=: 'float' and 'NoneType'

```

⚠ WARNING**print is not return**

Line 6 now **prints** the value instead of returning it. Look at the output: the first value, 1905.0, appears on screen, and then the program crashes.

- **print** shows a value **to you**. The program gets nothing back: the call gives **None**.
- **return** hands the value **to the program**. Line 11 needs it to add up.

Session 1's workshop hit the same wall with **round**. **A value on the screen is not a value in a variable.**

```

1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return deleted
13
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 tickers = ["AAPL", "NESN", "V00"]
18 shares = [10, 4, 2]
19 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21 total = total_value(tickers, shares, price)
22 print(f"total {total:,.2f}", round(total, 1))
23 print(years_to_double(), years_to_double(rate=0.07))
24 print(position_value(price=102.3, shares=4), sum(shares))
25 print(", ".join(tickers).lower(), type(total_value))

```

```
$ python portfolio.py
```

```
Traceback (most recent call last):
```

```
File "portfolio.py", line 22, in <module>
```

```
    print(f"total {total:,.2f}", round(total, 1))
          ~~~~~^~~~~~
```

```
TypeError: unsupported format string passed to NoneType.__format__
```

No return: the function gives back None

Line 12 is gone. The loop still adds up the total, inside the function. But nothing is sent back, so line 21 stores `None` in `total`.

Line 22 then tries to format `None` as money, and crashes.

Every function returns something. Without `return`, it returns `None`.

```

1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total, len(tickers)  <two values>
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  result = total_value(tickers, shares, price)
22  print(result, type(result))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))

```

```

$ python portfolio.py
(3338.2, 3) <class 'tuple'>  <one tuple>
15 11
409.2 16
aapl, nesn, voo <class 'function'>

```

Returning more than one value

On the left, line 12 now returns **two** values: the total and the number of positions.

line 12 **total** , **len(tickers)**

 ↘ ↙

one tuple **(3338.2, 3)**

return a, b packs the values into one **tuple** and sends it back. Line 22 shows it: still **one** return value, a **tuple** holding two.

```

1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total, len(tickers)
13
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 tickers = ["AAPL", "NESN", "V00"]
18 shares = [10, 4, 2]
19 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21 result = total_value(tickers, shares, price)
22 print(result, result[0], len(result))
23 print(years_to_double(), years_to_double(rate=0.07))
24 print(position_value(price=102.3, shares=4), sum(shares))
25 print(", ".join(tickers).lower(), type(total_value))

```

```

$ python portfolio.py
(3338.2, 3) 3338.2 2 result[0], len(result)
15 11
409.2 16
aapl, nesn, voo <class 'function'>

```

REMINDER

Tuples (session 3)

A **tuple** is like a list in round brackets, but it **cannot change** (immutable).

You write	Gives
result	(3338.2, 3)
result[0]	3338.2 : indexing, from 0
len(result)	2
result.append(4)	AttributeError : a tuple has no append

Tuples fit fixed groups of values: one position, one result of a function. Session 3 met them in `.items()` and `zip`.

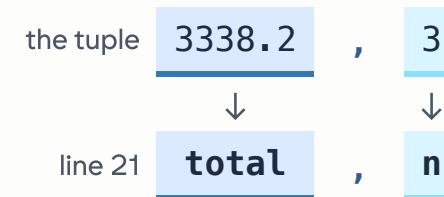
```

1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total, len(tickers)
13
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 tickers = ["AAPL", "NESN", "V00"]
18 shares = [10, 4, 2]
19 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21 total, n = total_value(tickers, shares, price)
22 one name per value total: {:.2f}".format(total, n)
23 print(years_to_double(), years_to_double(rate=0.07))
24 print(position_value(price=102.3, shares=4), sum(shares))
25 print(", ".join(tickers).lower(), type(total_value))

```

Unpacking: one name per value

Put as many names on the left as the tuple has values.
The first value goes to the first name, and so on:



One call, two results, two names. Too few or too many names gives a `ValueError`.

```

$ python portfolio.py
total 3,338.20 3 total and n
15 11
409.2 16
aapl, nesn, voo <class 'function'>

```

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

help() shows this

Docstrings and help()

help() shows a function's **signature** and its docstring:

```
def position_value(shares, price):
    """Value of one position, in CHF."""
    return shares * price

help(position_value)
```

Help on function position_value in module __main__:

```
position_value(shares, price)
    Value of one position, in CHF.
```

Every function in Python and in good libraries has one. In VS Code, **hover** over a name to see it.

```
$ python portfolio.py
```

```
total 3,338.20 3338.2
```

```
15 11
```

```
409.2 16
```

```
aapl, nesn, voo <class 'function'>
```

PART 3

Signatures and arguments

How the values in a call find their way to the names in a definition.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05): Signature, with a default
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

The signature: a function's contract

The **signature** is the first line of a definition: the name and its parameters. It tells you **how to call** the function.

Signature	Read it as
<code>position_value(shares, price)</code>	give me two values
<code>years_to_double(rate=0.05)</code>	give me a rate, or I use 0.05
<code>round(number, ndigits=None)</code>	a number, and optionally how many decimals

Two words people mix up: **parameters** are the names in the definition; **arguments** are the values in a call.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):  default
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers), years_to_double(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

Four kinds of arguments

Kind	In a call	Matched	On the left
positional	<code>f(a, b)</code>	by order	line 21
keyword	<code>f(b=2, a=1)</code>	by name	line 24
default	left out	the definition fills it in	line 14, <code>rate=0.05</code>
any number	<code>f(1, 2, 3, x=4)</code>	collected: <code>*args</code> , <code>**kwargs</code>	Good to know, later

The next slides take them one at a time.

```

1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):  Parameters
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 names = ["AAPL", "NESN", "V00"]
18 counts = [10, 4, 2]
19 quotes = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21 total = total_value(names, counts, quotes)  Arguments
22 print(f"total {total:,.2f}", round(total, 1))
23 print(years_to_double(), years_to_double(rate=0.07))
24 print(position_value(price=102.3, shares=4), sum(counts))
25 print(" ".join(names).lower(), type(total_value))

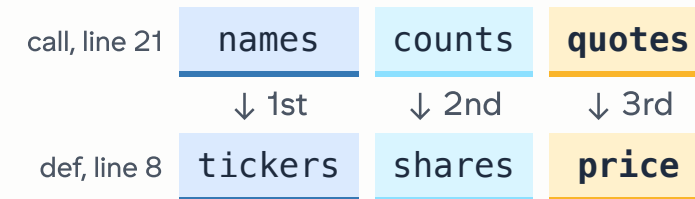
```

```

$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>

```

Positional arguments: matched by order



The **first** argument goes to the **first** parameter, and so on. Only the **order** counts.

On the left, the data now have **other names**: `names`, `counts`, `quotes`. The output does not change. Inside the function, the first argument is called `tickers`, whatever it was called outside.

portfolio.py **CHANGED ON THIS SLIDE**

```
1 # portfolio.py – the ledger, organised into functions
2 import math
3
4 def position_value(shares, price):
5     """Value of one position, in CHF."""
6     return shares * price
7
8 def total_value(tickers, shares, price):
9     total = 0.0
10    for i in range(len(tickers)):
11        total += position_value(shares[i], price[tickers[i]])
12    return total
13
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 tickers = ["AAPL", "NESN", "V00"]
18 shares = [10, 4, 2]
19 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21 total = total_value(shares, tickers, price)
22 print(f"total {total:,.2f}", round(total, 1))
23 print(years_to_double(), years_to_double(rate=0.07))
24 print(position_value(price=102.3, shares=4), sum(shares))
25 print(", ".join(tickers).lower(), type(total_value))
```

Swapped

```
$ python portfolio.py
```

```
Traceback (most recent call last):
```

```
File "portfolio.py", line 21, in <module>
    total = total_value(shares, tickers, price)
            ~~~~~
```

```
File "portfolio.py", line 11, in total_value
    total += position_value(shares[i], price[tickers[i]])
                        ~~~~~
```

```
KeyError: 10
```

⚠ WARNING

Swap two arguments

Line 21 now passes `shares` first and `tickers` second. Python does not know the names were meant the other way round: **it matches by position.**

Inside the function, `tickers` is now `[10, 4, 2]`, and line 11 looks up `price[10]`: `KeyError: 10`.

This time it crashed. With two numbers swapped, it would simply compute the wrong answer.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(Keyword arguments, years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
```

```
total 3,338.20 3338.2
```

```
15 11
```

```
409.2 16 4 × 102.3
```

```
aapl, nesn, voo <class 'function'>
```

Keyword arguments: matched by name

call, line 24

price=102.3

shares=4

↘ by name

↙ by name

def, line 4

shares

price

`name=value` in a call says **which parameter** gets the value. Order no longer matters: line 24 gives `price` first.

Keywords make a call **readable**: `sort_values("value", ascending=False)` says what `False` means.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05): Default
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
```

```
total 3,338.20 3338.2
```

```
15 11 default, then 7%
```

```
409.2 16
```

```
aapl, nesn, voo <class 'function'>
```

Default values: optional arguments

`rate=0.05` in the **definition** gives the parameter a **default**. A call can leave it out:

- `years_to_double()`: no argument, so `rate` is `0.05`. **15** years.
- `years_to_double(rate=0.07)`: the call overrides the default. **11** years.

Library functions have many parameters with defaults. You usually set **one or two**, by keyword, and leave the rest.

```

1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, 4), sum(shares))
25  print(", ".join(tickers), total_value(shares, price))

```

Keyword, then positional

```
$ python portfolio.py
```

```
File "portfolio.py", line 24
```

```
print(position_value(price=102.3, 4), sum(shares))
      ^
```

```
SyntaxError: positional argument follows keyword argument
```

⚠ WARNING

The rule: positional first, keyword after

Line 24 now has a keyword argument **before** a positional one. Python refuses to start: `SyntaxError`.

Once you name one argument, Python can no longer tell which position the next one fills. So: **all positional arguments first, then keywords**.

portfolio.py **CHANGED ON THIS SLIDE**

```
1 # portfolio.py – the ledger, organised into functions
2 import math
3
4 def position_value(shares, price):
5     """Value of one position, in CHF."""
6     return shares * price
7
8 def total_value(tickers, shares, price):
9     total = 0.0
10    for i in range(len(tickers)):
11        total += position_value(shares[i], price[tickers[i]])
12    return total
13
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 tickers = ["AAPL", "NESN", "V00"]
18 shares = [10, 4, 2]
19 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21 total = total_value(tickers, shares, price)
22 print(f"total {total:,.2f}", round(total, 1))
23 print(years_to_double(), years_to_double(rate=0.07))
24 print(position_value(4), sum(shares))
25 print(", ".join(tickers).lower(), type(total_value))
```

One argument, two needed

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
Traceback (most recent call last):
  File "portfolio.py", line 24, in <module>
    print(position_value(4), sum(shares))
          ~~~~~
TypeError: position_value() missing 1 required positional argument: 'price'
```

Too few arguments

`position_value` needs two values. Line 24 now gives one.

The error message is exact: **which** function, **what** is missing, and its **name**: missing 1 required positional argument: 'price'.

Read it like a signature check: the call did not match the contract.

```

1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(r=0.07))
24  print(position_value(price=102.3, shares=
25  print(", ".join(tickers).lower(), blue(total_value))

```

No parameter called r

```

$ python portfolio.py
total 3,338.20 3338.2
Traceback (most recent call last):
  File "portfolio.py", line 23, in <module>
    print(years_to_double(), years_to_double(r=0.07))
    ~~~~~
TypeError: years_to_double() got an unexpected keyword argument 'r'

```

A keyword that does not exist

Line 23 now says `r=0.07`. The parameter is called `rate`, so Python refuses: unexpected keyword argument `'r'`.

When a library call fails like this, **check the signature:** `help(...)`, or hover in VS Code. The fix is almost always a spelling.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

+ GOOD TO KNOW

Reading a real signature

`help(sorted)` begins with:

```
sorted(iterable, /, *, key=None, reverse=False)
```

- `iterable`: one positional argument, the thing to sort.
- `/` and `*` are markers. After `*`, arguments **must** be given by keyword.
- `reverse=False`: a default. To change it, name it:

```
shares = [10, 4, 2]
print(sorted(shares), sorted(shares, reverse=True))
```

```
[2, 4, 10] [10, 4, 2]
```

You do not need to write signatures like this. You will **read** them in every library.

```

1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def report(*names, **options):  <collect everything>
15     print(names, options)
16
17     tickers = ["AAPL", "NESN", "V00"]
18     shares = [10, 4, 2]
19     price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21     total = total_value(tickers, shares, price)
22     print(f"total {total:,.2f}", round(total, 1))
23     report("AAPL", "NESN", currency="CHF", digits=2)
24     print(position_value(price=102.3, shares=4), sum(shares))
25     print(", ".join(tickers).lower(), type(total_value))

```

```

$ python portfolio.py
total 3,338.20 3338.2
('AAPL', 'NESN') {'currency': 'CHF', 'digits': 2} <a tuple>
409.2 16
aapl, nesn, voo <class 'function'>

```

+ GOOD TO KNOW***args and **kwargs: any number of arguments**

* collects extra **positional** arguments into a tuple, ** extra **keyword** arguments into a dict. `print(*args, sep=" ")` works this way.

You will see it **a lot in AI-generated code**. Read it as "takes whatever it is given".

PART 4

Scope: whose names are these?

Each call gets its own set of names, and throws them away when it returns.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0  # total inside the function
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20  # a different total
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

Two variables called `total`

Global names (the whole file)

`math`, `position_value`,
`total_value`,
`years_to_double`, `tickers`,
`shares`, `price`, `total` (line 21)

Local names (one call of `total_value`)

`tickers`, `shares`, `price` (the parameters), `total` (line 9),
`i`

A name created **inside** a function is **local**: it lives only during that call. The `total` of line 9 and the `total` of line 21 are **two different variables** that share a name.

Line 21 only gets the local `total`'s **value**, through `return`.

```

1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price * fx  < not local: found outside
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14     def years_to_double(rate=0.05):
15         return math.ceil(math.log(2) / math.log(1 + rate))
16     fx = 1.0  < a global
17     tickers = ["AAPL", "NESN", "V00"]
18     shares = [10, 4, 2]
19     price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21     total = total_value(tickers, shares, price)
22     print(f"total {total:,.2f}", round(total, 1))
23     print(years_to_double(), years_to_double(rate=0.07))
24     print(position_value(price=102.3, shares=4), sum(shares))
25     print(", ".join(tickers).lower(), type(total_value))

```

```

$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>

```

+ GOOD TO KNOW

Can a function see names from outside?

Reading: yes. On the left, line 6 uses `fx`, created on line 16, outside every function. Python looks for a name in the **local** box first, then in the **global** box, then among the built-ins. The output is unchanged.

Assigning: no. `total = 0.0` on line 9 does **not** touch a global `total`: it creates a new **local** one, which **hides** any global of the same name inside the function. That is why the two `total`s on the previous slide never mix.

Reading globals works, but it hides what a function depends on. Pass values in as **arguments**.

portfolio.py **CHANGED ON THIS SLIDE**

```
1 # portfolio.py – the ledger, organised into functions
2 import math
3
4 def position_value(shares, price):
5     """Value of one position, in CHF."""
6     return shares * price
7
8 def total_value(tickers, shares, price):
9     total = 0.0
10    for i in range(len(tickers)):
11        total += position_value(shares[i], price[tickers[i]])
12    return total
13
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 tickers = ["AAPL", "NESN", "V00"]
18 shares = [10, 4, 2]
19 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21 total = total_value(tickers, shares, price)
22 print(f"total {total:,.2f}", round(total, 1))
23 print(years_to_double(), years_to_double(rate=0.07))
24 print(position_value(price=102.3, shares=4), sum(shares))
25 print(i)  ← i was local to total_value
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
Traceback (most recent call last):
  File "portfolio.py", line 25, in <module>
    print(i)
    ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

Local names disappear after the call

`i` was created by the loop on line 10, **inside** `total_value`. When the function returned, its local names were thrown away.

So line 25 cannot see `i`: `NameError: name 'i' is not defined`.

To get something out of a function, **return** it. There is no other clean way out.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

Parameters are local names too

Inside `total_value`, `tickers` is the **parameter**: whatever the call handed over. It is not the global `tickers` from line 17, even though here they hold the same list.

Call `total_value(other_list, other_shares, other_prices)` and the function works on those instead. That is what makes it **reusable**.

Good habit: a function uses **its parameters**, not global variables. Everything it needs comes in through the parentheses.

```

1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     shares.append(3)  < Changes the caller's list
13     return total
14
15 def years_to_double(rate=0.05):
16     return math.ceil(math.log(2) / math.log(1 + rate))
17
18 tickers = ["AAPL", "NESN", "V00"]
19 shares = [10, 4, 2]
20 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
21
22 total = total_value(tickers, shares, price)
23 print(f"total {total:,.2f}", round(total, 1))
24 print(years_to_double(), years_to_double(rate=0.07))
25 print(position_value(price=102.3, shares=4), sum(shares))
26 print(", ".join(tickers).lower(), type(total_value))

```

```

$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 19  < was 16: now 19
aapl, nesn, voo <class 'function'>

```

⚠ WARNING

A function can change the list you give it

On the left, `total_value` now also does `shares.append(3)` before returning.

Inside the function, `shares` is a **second name for the same list** as the global `shares` (session 3: two names, one list). So the caller's list changed: line 24's `sum(shares)` now gives **19**, not 16. The total itself is still right, which makes this easy to miss.

Say in the docstring if a function changes its argument, or return a new value instead.

```
portfolio.py
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

+ GOOD TO KNOW

The technical words: mutable and immutable

	Can change after creation?	Types
mutable	yes: <code>.append</code> , <code>item assignment</code> , <code>.sort()</code>	<code>list</code> , <code>dict</code> , <code>numpy ndarray</code> , <code>pandas DataFrame</code>
immutable	no: every operation makes a new value	<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code> , <code>tuple</code> , <code>None</code>

Passing an **immutable** value to a function is always safe: nothing the function does can change it. Passing a **mutable** one hands over the object itself.

You will meet these two words in documentation, error messages and AI explanations.

PART 5

Methods

Functions that belong to a type,
written after a dot.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.05))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

Two ways to write a call

	Written	Example	Belongs to
function	f(value) : the value goes inside	sum(shares) , len(ticker)	no one type: works on many
method	value.f() : the value comes before the dot	arr.sum() , ticker.lower()	one type: ndarray , str

A **method** is a function that belongs to a type.

`value.method(arguments)` calls it on that value: the value before the dot is handed over automatically.

`sum(shares)` and `arr.sum()` can give the same 16.

They are different pieces of code: a built-in function, and a method of numpy's array.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print("Two methods in a row", years_to_double(rate=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
```

```
total 3,338.20 3338.2
```

```
15 11
```

```
409.2 16
```

```
aapl, nesn, voo <class 'function'> the result
```

Methods in a chain: read left to right

line 25 `", ".join(tickers).lower()`

↓ join: a str method

`"AAPL, NESN, V00".lower()`

↓ lower: a str method

result `"aapl, nesn, voo"`

Each method returns a **new value**, and the next method is called on it. Chains are everywhere in pandas code.

`join` belongs to the **separator** string: `", "` joins the list's items with a comma between them.

portfolio.py **CHANGED ON THIS SLIDE**

```
1 # portfolio.py – the ledger, organised into functions
2 import math
3
4 def position_value(shares, price):
5     """Value of one position, in CHF."""
6     return shares * price
7
8 def total_value(tickers, shares, price):
9     total = 0.0
10    for i in range(len(tickers)):
11        total += position_value(shares[i], price[tickers[i]])
12    return total
13
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 tickers = ["AAPL", "NESN", "V00"]
18 shares = [10, 4, 2]
19 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21 total = total_value(tickers, shares, price)
22 print(f"total {total:,.2f}", round(total, 1))
23 print(years_to_double(), years_to_double(rate=0.07))
24 print(position_value(price=102.3, shares=4), shares.sum())
25 print(", ".join(tickers).lower(), type(total))
```

list has no .sum

Methods belong to a type

Line 24 now calls `shares.sum()`. `shares` is a **list**, and lists have no `sum` method: `AttributeError: 'list' object has no attribute 'sum'`.

A numpy array has `.sum()`. A list does not. Before you call a method, know the **type** of the value in front of the dot (session 3's rule, again).

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
Traceback (most recent call last):
  File "portfolio.py", line 24, in <module>
    print(position_value(price=102.3, shares=4), shares.sum())
    ~~~~~~
AttributeError: 'list' object has no attribute 'sum'
```

```
1 # portfolio.py – the ledger, organised into functions
2 import math
3
4 def position_value(shares, price):
5     """Value of one position, in CHF."""
6     return shares * price
7
8 def total_value(tickers, shares, price):
9     total = 0.0
10    for i in range(len(tickers)):
11        total += position_value(shares[i], price[tickers[i]])
12    return total
13
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 tickers = ["AAPL", "NESN", "V00"]
18 shares = [10, 4, 2]
19 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21 total = total_value(tickers, shares, price)
22 print(f"total {total:,.2f}", round(total, 1))
23 print(years to double(). years_to_double(rate=0.07))
24 print(Changes shares, returns None 3, shares=4), sum(shares))
25 print(shares.append(5))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
None
```

⚠ WARNING

Some methods change the value, some return a new one

Line 25 now prints the result of `shares.append(5)` : `None` .

Method	Changes the value?	Returns
<code>ticker.lower()</code>	no: strings never change	a new string
<code>shares.append(5)</code>	yes, in place	None
<code>shares.sort()</code>	yes, in place	None

So `shares = shares.append(5)` destroys your list: `shares` becomes `None` . Know which kind of method you are calling.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 tickers = ["AAPL", "NESN", "V00"]
18 shares = [10, 4, 2]
19 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21 total = total_value(tickers, shares, price)
22 print(f"total {total:,.2f}", round(total, 1))
23 print(years_to_double(), years_to_double(rate=0.07))
24 print(position_value(price=102.3, shares=4), sum(shares))
25 print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

Methods you will use every week

Type	Methods
str	<code>.lower()</code> <code>.upper()</code> <code>.strip()</code> <code>.split(",")</code> <code>.replace(a, b)</code> <code>", ".join(items)</code> <code>.startswith("N")</code>
list	<code>.append(x)</code> <code>.sort()</code> <code>.copy()</code> <code>.index(x)</code> <code>.count(x)</code>
dict	<code>.get(k, default)</code> <code>.keys()</code> <code>.values()</code> <code>.items()</code>

```
line = "  NESN,4,102.30  "
print(line.strip().split(","), "NESN".startswith("N"), "a-b".replace("-", "+"))
```

```
['NESN', '4', '102.30'] True a+b
```

To see all methods of a value: type the value, then a dot, in VS Code, and wait for the list.

PART 6

Modules and **import**

Functions other people wrote, ready to use.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math  ← Load the module
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16         ← module.function
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

A module is a file full of functions

math a module

- └─ ceil round up
- └─ log logarithm
- └─ sqrt square root
- └─ pi 3.14159...

`import math` (line 2) loads the module once. After that, `math.ceil(...)` means **"the function `ceil` inside `math`"**.

The dot again: this time it means **"inside"**. A module keeps its names in their own box, so `math.log` never clashes with a `log` of yours.

```

1  # portfolio.py – the ledger, organised into functions
2  from math import ceil, log  ← Import just these names
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return ceil(log(2) / log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))

```

```

$ python portfolio.py
total 3,338.20 3338.2
15 11 ← Same result
409.2 16
aapl, nesn, voo <class 'function'>

```

Three ways to import

What each line puts **into your file's names:**

You write

Your file now has

import math

→

math ceil · log · sqrt · pi ...

call math.ceil(x)

from math import ceil, log

→

ceil log

only these two, no box: call

ceil(x)

import numpy as np

→

np array · mean · sum ...

a short nickname:

np.array(x)

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
```

```
total 3,338.20 3338.2
```

```
15 11
```

```
409.2 16
```

```
aapl, nesn, voo <class 'function'>
```

The standard library: already installed

Python comes with hundreds of modules. A few you will meet:

Module	For
math	logs, square roots, rounding up and down
statistics	mean, median, standard deviation
random	random numbers, shuffling, sampling
datetime	dates and times
csv, pathlib	reading files and finding them (session 6)

```
import statistics
prices = [190.5, 102.3, 512.0]
print(statistics.mean(prices), statistics.median(prices))
```

```
268.26666666666665 190.5
```

```

1  # portfolio.py – the ledger, organised into functions
2  import math, random
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14     def years_to_double(rate=0.05):
15         return math.ceil(math.log(2) / math.log(1 + rate))
16     random.seed(42)  ← The seed
17     tickers = ["AAPL", "NESN", "V00"]
18     shares = [10, 4, 2]
19     price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21     total = total_value(tickers, shares, price)
22     print(f"total {total:,.2f}", round(total, 1))
23     print(years_to_double(), years_to_double(rate=0.07))
24     print(roll_a_die_value(price=102.3, shares=4), sum(shares))
25     print(random.randint(1, 6), random.randint(1, 6))

```

```

$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16

```

6 1 ← Two rolls

random: numbers that look random

On the left, the script imports `random` and, on line 25, rolls a die twice: `random.randint(1, 6)` gives a whole number from 1 to 6.

Random numbers are how we **simulate**: markets, surveys, games. The survey chart in session 1 came from such a simulation.

But a computer cannot really roll a die. Line 16 is the clue: `random.seed(42)`.

```

1  # portfolio.py – the ledger, organised into functions
2  import math, random
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16  random.seed(7)  ← Another seed
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(random.randint(1, 6), random.randint(1, 6))

```

```

$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
3 2 ← Other rolls, every time

```

Seeds and determinism

The "random" numbers are **computed**, step by step, from a starting number: the **seed**.

Seed	Rolls	Run it again
random.seed(42)	6 1	6 1
random.seed(7)	3 2 (on the left)	3 2

A program is **deterministic** when the same code and the same input always give the same output. With a seed, even a simulation is deterministic: everyone gets the same numbers, and a result can be checked. Every simulation in this course sets a seed.

```

1  # portfolio.py – the ledger, organised into functions
2  import maths  <No such module>
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))

```

```
$ python portfolio.py
```

```
Traceback (most recent call last):
```

```
  File "portfolio.py", line 2, in <module>
    import maths
```

```
ModuleNotFoundError: No module named 'maths'
```

When an import fails

Line 2 now says `maths`. There is no module with that name: `ModuleNotFoundError`, and the program stops at line 2.

Two usual causes:

- a **typo** in the name, as here;
- a **library that is not installed**. On Nuvolos, numpy and pandas are installed for everyone; installing new ones is not part of this course.

PART 7

Classes and objects

Why every type comes with its own
operations.

objects.py

```
1  # objects.py – every value is an object of some class
2  import numpy as np
3  import pandas as pd
4
5  shares = [10, 4, 2]  ← an object
6  arr = np.array(shares)  ← another object
7  print(sum(shares), arr.sum(), arr.mean())
8  print(type(shares), type(arr))
9  print(arr.shape, arr.dtype, len(arr))
10
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "V00"],
12                        "shares": shares,
13                        "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger["value"].sum(), ledger.shape)
16 print(type(ledger), type(ledger["value"]))
17 print(ledger.sort_values("value", ascending=False).head(2))
```

```
$ python objects.py
```

```
16 16 5.333333333333333
<class 'list'> <class 'numpy.ndarray'>
(3,) int64 3
3338.2 (3, 4)
<class 'pandas.DataFrame'> <class 'pandas.Series'>
   ticker  shares  price  value
0    AAPL      10   190.5  1905.0
2     V00       2   512.0  1024.0
```

An object: a value with its own tools

In Python, every value is an **object**: a piece of **data**, packed together with the **operations** that make sense for it.

ticker = "NESN"

DATA

N E S N

TOOLS

.lower() .upper() .split()

...

shares = [10, 4, 2]

DATA

10 · 4 · 2

TOOLS

.append() .sort() .copy()

...

The dot opens the toolbox: `ticker.lower()` means "use the `lower` tool of this object". You have done this since session 3.

objects.py

```
1  # objects.py – every value is an object of some class
2  import numpy as np
3  import pandas as pd
4
5  shares = [10, 4, 2]
6  arr = np.array(shares)
7  print(sum(shares), arr.sum(), arr.mean())
8  print(type(shares), type(arr))
9  print(arr.shape, arr.dtype, len(arr))
10
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "V00"],
12                        "shares": shares,
13                        "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger["value"].sum(), ledger.shape)
16 print(type(ledger), type(ledger["value"]))
17 print(ledger.sort_values("value", ascending=False).head(2))
```

```
$ python objects.py
```

```
16 16 5.333333333333333
```

```
<class 'list'> <class 'numpy.ndarray'> type() names the class
```

```
(3,) int64 3
```

```
3338.2 (3, 4)
```

```
<class 'pandas.DataFrame'> <class 'pandas.Series'>
```

```
   ticker  shares  price  value
0   AAPL      10   190.5  1905.0
2    V00       2   512.0  1024.0
```

A class: the kind of object

Every list has the same tools. Something must define them once, for all lists: that is the **class** `list`.

class `list`

defined once: what every list holds, and which tools it has



```
[10, 4, 2]
```

```
[190.5, 102.3]
```

```
[]
```

Each list is an **object of the class** `list` (also called an **instance**). `type(x)` names the class: `<class 'list'>`. So a **type** and a **class** are the same thing, and session 2's rule, "a type comes with its own operations", simply says: **the class decides the tools**.

objects.py

```
1 # objects.py – every value is an object of some class
2 import numpy as np
3 import pandas as pd
4
5 shares = [10, 4, 2]
6 arr = np.array(shares)
7 print(sum(shares), arr.sum(), arr.mean()) Methods: ()
8 print(type(shares), type(arr))
9 print(arr.shape, arr.dtype, len(arr))
10 Attributes: no ()
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "V00"],
12                        "shares": shares,
13                        "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger["value"].sum(), ledger.shape)
16 print(type(ledger), type(ledger["value"]))
17 print(ledger.sort_values("value", ascending=False).head(2))
```

```
$ python objects.py
```

```
16 16 5.333333333333333
<class 'list'> <class 'numpy.ndarray'>
(3,) int64 3
3338.2 (3, 4)
<class 'pandas.DataFrame'> <class 'pandas.Series'>
  ticker  shares  price  value
0  AAPL      10  190.5  1905.0
2   V00       2  512.0  1024.0
```

numpy's ndarray: attributes and methods

`np.array(shares)` creates an object of the class `numpy.ndarray`. It carries two kinds of things after the dot:

	Written	On the left	Gives
attribute: a stored fact	no parentheses	<code>arr.shape</code> , <code>arr.dtype</code>	<code>(3,)</code> , <code>int64</code>
method: an operation	parentheses	<code>arr.sum()</code> , <code>arr.mean()</code>	<code>16</code> , <code>5.33...</code>

`shape` is the array's size, `dtype` the one type all its elements share (session 3). An attribute is **looked up**; a method is **called**.

objects.py

```
1 # objects.py – every value is an object of some class
2 import numpy as np
3 import pandas as pd
4
5 shares = [10, 4, 2]
6 arr = Same 16, two routes
7 print(sum(shares), arr.sum(), arr.mean())
8 print(type(shares), type(arr))
9 print(arr.shape, arr.dtype, len(arr))
10
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "V00"],
12                          "shares": shares,
13                          "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger["value"].sum(), ledger.shape)
16 print(type(ledger), type(ledger["value"]))
17 print(ledger.sort_values("value", ascending=False).head(2))
```

sum(shares) and arr.sum()

built-in function	sum(shares)	→	16
ndarray method	arr.sum()	→	16

Same answer, different code: `sum` is Python's built-in function and works on any list of numbers; `.sum()` is written by numpy, for arrays, and is much faster on millions of numbers.

Rule of thumb: on a **list**, use the function. On an **array** or a **pandas** table, use the method.

```
$ python objects.py
```

```
16 16 5.333333333333333 list + function · array + method
<class 'list'> <class 'numpy.ndarray'>
(3,) int64 3
3338.2 (3, 4)
<class 'pandas.DataFrame'> <class 'pandas.Series'>
  ticker  shares  price  value
0  AAPL      10  190.5  1905.0
2   V00       2  512.0  1024.0
```

objects.py

```
1  # objects.py – every value is an object of some class
2  import numpy as np
3  import pandas as pd
4
5  shares = [10, 4, 2]
6  arr = np.array(shares)
7  print(sum(shares), arr.sum(), arr.mean())
8  print(type(shares), type(arr))
9  print(arr.shape, arr.dtype, len(arr))
10
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "V00"],
12                        "shares": shares,
13                        "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger["value"].sum(), ledger.shape)
16 print(type(ledger), type(ledger["value"]))
17 print(ledger.sort_values("value", ascending=False).head(2))
```

```
$ python objects.py
```

```
16 16 5.333333333333333
```

```
<class 'list'> <class 'numpy.ndarray'>
```

```
(3,) int64 3
```

```
3338.2 (3, 4)
```

```
<class 'pandas.DataFrame'> <class 'pandas.Series'>
```

Two pandas classes

		price	value
0	AAPL	10	1905.0
2	V00	2	1024.0

pandas: DataFrame and Series

A pandas table is an object of the class `DataFrame`. One of its columns, `ledger["value"]`, is an object of another class: `Series`.

- `ledger.shape` is an attribute: `(3, 4)`, three rows and four columns.
- `ledger["value"].sum()` is a method of `Series`: `3338.2`, session 3's total once more.

Two classes, each with **hundreds** of methods. Part 2 of the course lives among them.

Older pandas prints `pandas.core.frame.DataFrame`. Same class.

objects.py

```
1  # objects.py – every value is an object of some class
2  import numpy as np
3  import pandas as pd
4
5  shares = [10, 4, 2]
6  arr = np.array(shares)
7  print(sum(shares), arr.sum(), arr.mean())
8  print(type(shares), type(arr))
9  print(arr.shape, arr.dtype, len(arr))
10
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "VOO"],
12                        "shares": shares,
13                        "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger["value"].sum(), ledger.shape)
16 print(type(ledger["value"]), ledger["value"])
17 print(ledger.sort_values("value", ascending=False).head(2))
```

positional + keyword

then another
method

```
$ python objects.py
```

```
16 16 5.333333333333333
<class 'list'> <class 'numpy.ndarray'>
(3,) int64 3
3338.2 (3, 4)
<class 'pandas.DataFrame'> <class 'pandas.Series'>
   ticker  shares  price  value
0    AAPL     10  190.5  1905.0
2     VOO      2  512.0  1024.0
```

Everything from today, in one line

```
ledger.sort_values("value", ascending=False).head(2)
```

- `ledger.sort_values(...)` is a **method** of the `DataFrame` object.
- `"value"` is a **positional** argument: the column to sort by.
- `ascending=False` is a **keyword** argument, overriding a **default**.
- `.head(2)` is called on the **new** table that `sort_values` returned: a **chain**.

The two largest positions, AAPL and VOO. Line 17 now reads as easily as a sentence.

objects.py

```
1  # objects.py – every value is an object of some class
2  import numpy as np
3  import pandas as pd
4
5  shares = [10, 4, 2]
6  arr = np.array(shares)
7  print(sum(shares), arr.sum(), arr.mean())
8  print(type(shares), type(arr))
9  print(arr.shape, arr.dtype, len(arr))
10
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "V00"],
12                        "shares": shares,
13                        "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger["value"].sum(), ledger.shape)
16 print(type(ledger), type(ledger["value"]))
17 print(ledger.sort_values("value", ascending=False).head(2))
```

```
$ python objects.py
```

```
16 16 5.333333333333333
<class 'list'> <class 'numpy.ndarray'>
(3,) int64 3
3338.2 (3, 4)
<class 'pandas.DataFrame'> <class 'pandas.Series'>
   ticker  shares  price  value
0    AAPL      10   190.5  1905.0
2     V00       2   512.0  1024.0
```

Why classes?

A class bundles **data** and the **operations that make sense for it**:

- a `str` knows how to `.lower()` itself; a number does not;
- a numpy `ndarray` knows how to add element by element (session 3);
- a pandas `DataFrame` knows how to sort, group and plot its rows.

When you use a library, you mostly use **its classes**: you create objects, then call their methods. `type(x)` tells you which class you are dealing with, and so **which operations exist**.

positions.py

```
1  # positions.py – our own class for one position of the ledger
2  class Position:
3      """One position: a ticker, a number of shares, a price."""
4
5      def __init__(self, ticker, shares, price):
6          self.ticker = ticker
7          self.shares = shares
8          self.price = price
9
10     def value(self):
11         return self.shares * self.price
12
13     nesn = Position("NESN", 4, 102.3)
14     aapl = Position("AAPL", 10, 190.5)
15     print(nesn.ticker, nesn.shares, nesn.value())
16     print(aapl.ticker, aapl.value())
17     print(
18         every value has a class
19         nesn, Position)
20     print(
21         type(4), type("NESN"), type([10, 4, 2]))
```

```
$ python positions.py
```

```
NESN 4 409.2
```

```
AAPL 1905.0
```

```
<class '__main__.Position'> True
```

```
<class 'int'> <class 'str'> <class 'list'>
```

instances of int, str, list

Everything is an object, so everything is an instance

In Python, **every value is an object**. And every object is an **instance** of some class:

- 4 is an instance of `int`, "NESN" of `str`, [10, 4, 2] of `list`;
- `arr` is an instance of `numpy.ndarray`, `ledger` of `pandas.DataFrame`.

The classes above were written by Python's and the libraries' authors. On the left, a new script: **we write a class of our own**, `Position`. You will meet classes like it constantly in other people's code, and in AI-generated code.

positions.py

```
1  # positions.py – our own class for one position of the ledger
2  class Position:  # the class
3      """One position: a ticker, a number of shares, a price."""
4
5      def __init__(self, ticker, shares, price):  # the constructor
6          self.ticker = ticker
7          self.shares = shares  # attributes
8          self.price = price
9
10     def value(self):  # a method
11         return self.shares * self.price
12
13     nesn = Position("NESN", 4, 102.3)
14     aapl = Position("AAPL", 10, 190.5)
15     print(nesn.ticker, nesn.shares, nesn.value())
16     print(aapl.ticker, aapl.value())
17     print(type(nesn), isinstance(nesn, Position))
18     print(type(4), type("NESN"), type([10, 4, 2]))
```

Anatomy of a class

Lines	Code	What it is
2	class Position:	a new class: a new type
3	"""One position..."""	docstring
5–8	def __init__(self, ...):	the constructor
6–8	self.ticker = ticker	attributes: the data
10–11	def value(self):	a method: a tool

A class is a **blueprint**: it says what data every position holds (lines 5–8), and which tools every position has (lines 10–11). Methods are ordinary functions, written **inside** the class.

```
$ python positions.py
```

```
NESN 4 409.2
```

```
AAPL 1905.0
```

```
<class '__main__.Position'> True
```

```
<class 'int'> <class 'str'> <class 'list'>
```

positions.py

```
1  # positions.py – our own class for one position of the ledger
2  class Position:
3      """One position: a ticker, a number of shares, a price."""
4
5      def __init__(self, ticker, shares, price):  a dunder
6          self.ticker = ticker
7          self.shares = shares
8          self.price = price
9
10     def value(self):
11         return self.shares * self.price
12
13     nesn = Position("NESN", 4, 102.3)
14     aapl = Position("AAPL", 10, 190.5)
15     print(nesn.ticker, nesn.shares, nesn.value())
16     print(aapl.ticker, aapl.value())
17     print(type(nesn), isinstance(nesn, Position))
18     print(type(4), type("NESN"), type([10, 4, 2]))
```

```
$ python positions.py
```

```
NESN 4 409.2
AAPL 1905.0
<class '__main__.Position'> True
<class 'int'> <class 'str'> <class 'list'>
```

+ GOOD TO KNOW

Dunders: `__double underscore__` names

`__init__` is a **dunder** (double **underscore**). Python calls dunder **for you**:

You write	Python calls	Gives
<code>Position(...)</code>	<code>__init__</code>	a new instance
<code>len("NESN")</code>	<code>"NESN".__len__()</code>	4
<code>3 + 4</code>	<code>(3).__add__(4)</code>	7
<code>print(nesn)</code>	<code>__repr__</code> / <code>__str__</code>	how the object prints

Many people find dunder confusing at first. But they are **everywhere**, especially in **AI-generated code**. Read one as "Python calls this when..."

positions.py

```
1  # positions.py – our own class for one position of the ledger
2  class Position:
3      """One position: a ticker, a number of shares, a price."""
4
5      def __init__(self, ticker, shares, price):
6          self.ticker = ticker
7          self.shares = shares
8          self.price = price
9
10     def value(self):
11         return self.shares * self.price
12
13     nesn = Position("NESN", 4, 102.3)
14     aapl = Position("AAPL", 10, 190.5)
15     print(nesn.ticker, nesn.shares, nesn.value())
16     print(aapl.ticker, aapl.value())
17     print(type(nesn), isinstance(nesn, Position))
18     print(type(4), type("NESN"), type([10, 4, 2]))
```

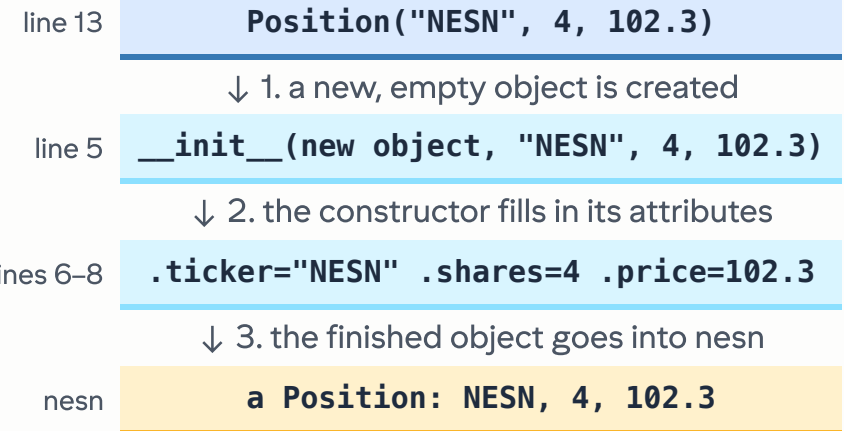
self = the new object

call the class

```
$ python positions.py
NESN 4 409.2
AAPL 1905.0
<class '__main__.Position'> True
<class 'int'> <class 'str'> <class 'list'>
```

The constructor: making an instance

Calling the class like a function **constructs** a new object:



`__init__` (two underscores on each side) is the constructor's fixed name. You never call it directly: calling the class does.

positions.py

```
1  # positions.py – our own class for one position of the ledger
2  class Position:
3      """One position: a ticker, a number of shares, a price."""
4
5      def __init__(self, ticker, shares, price):
6          self.ticker = ticker
7          self.shares = shares
8          self.price = price
9
10     def value(self):
11         return self.shares * self.price
12
13     nesn = Position("NESN", 4, 102.3)  ← instance 1
14     aapl = Position("AAPL", 10, 190.5) ← instance 2
15     print(nesn.ticker, nesn.shares, nesn.value())
16     print(aapl.ticker, aapl.value())
17     print(type(nesn), isinstance(nesn, Position))
18     print(type(4), type("NESN"), type([10, 4, 2]))
```

Many instances, one class

class
Position
defined once:
attributes ticker,
shares, price
method value()



nesn
ticker =
"NESN"
shares = 4
price = 102.3

aapl
ticker =
"AAPL"
shares = 10
price = 190.5

Each **instance** keeps its **own** attribute values:

`nesn.shares` is 4, `aapl.shares` is 10. They **share** the class's methods: both can call `.value()`.

Attributes are read with a dot and no parentheses (`nesn.ticker`), like numpy's `arr.shape`.

```
$ python positions.py
```

```
NESN 4 409.2  ← nesn's own data
```

```
AAPL 1905.0
```

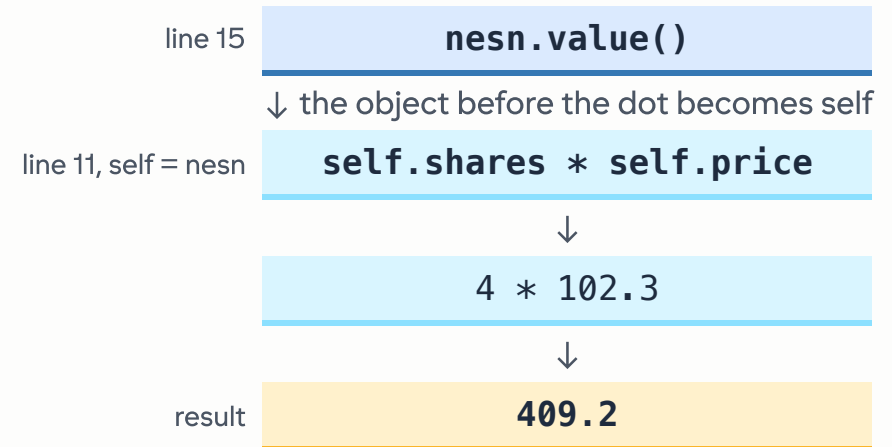
```
<class '__main__.Position'> True
```

```
<class 'int'> <class 'str'> <class 'list'>
```

positions.py

```
1  # positions.py – our own class for one position of the ledger
2  class Position:
3      """One position: a ticker, a number of shares, a price."""
4
5      def __init__(self, ticker, shares, price):
6          self.ticker = ticker
7          self.shares = shares
8          self.price = price
9
10     def value(self):
11         return self.shares * self.price  < 4 * 102.3
12
13     nesn = Position("NESN", 4, 102.3)
14     aapl = Position("AAPL", 10, 190.5)
15     print(nesn.ticker, nesn.shares, nesn.value())  < self is nesn
16     print(aapl.ticker, aapl.value())
17     print(type(nesn), isinstance(nesn, Position))
18     print(type(4), type("NESN"), type([10, 4, 2]))
```

self: the object the method was called on



Every method's first parameter is `self`, but you never pass it: Python hands over the object in front of the dot. That is exactly what happened with `ticker.lower()` and `arr.sum()` all along.

```
$ python positions.py
```

```
NESN 4 409.2 nesn.value()
```

```
AAPL 1905.0
```

```
<class '__main__.Position'> True
```

```
<class 'int'> <class 'str'> <class 'list'>
```

positions.py

```
1  # positions.py – our own class for one position of the ledger
2  class Position:
3      """One position: a ticker, a number of shares, a price."""
4
5      def __init__(self, ticker, shares, price):
6          self.ticker = ticker
7          self.shares = shares
8          self.price = price
9
10     def value(self):
11         return self.shares * self.price
12
13     nesn = Position("NESN", 4, 102.3)
14     aapl = Position("AAPL", 10, 190.5)
15     print(nesn.ticker, nesn.shares, nesn.value())
16     print(aapl.ticker, aapl.value())
17     print(type(nesn), isinstance(nesn, Position))
18     print(type(4), type("NESN"), type([10, 4, 2]))
```

Our class is a real type

`type(nesn)` prints `<class '__main__.Position'>`:
`Position` is now a type like `int` or `list` (`__main__` means "defined in the script we ran").

`isinstance(nesn, Position)` asks "is this object an instance of that class?": `True`.

A class is how you make a new type. numpy's `ndarray` and pandas' `DataFrame` are classes written exactly like this, only much larger.

```
$ python positions.py
```

```
NESN 4 409.2
```

```
AAPL 1905.0
```

```
<class '__main__.Position'> True
```

a new type

```
<class 'int'> <class 'str'> <class 'list'>
```

positions.py

```
1  # positions.py – our own class for one position of the ledger
2  class Position:
3      """One position: a ticker, a number of shares, a price."""
4
5      def __init__(self, ticker, shares, price):
6          self.ticker = ticker
7          self.shares = shares
8          self.price = price
9
10     def value(self):
11         return self.shares * self.price
12
13     nesn = Position("NESN", 4, 102.3)
14     aapl = Position("AAPL", 10, 190.5)
15     print(nesn.ticker, nesn.shares, nesn.value())
16     print(aapl.ticker, aapl.value())
17     print(type(nesn), isinstance(nesn, Position))
18     print(type(4), type("NESN"), type([10, 4, 2]))
```

```
$ python positions.py
```

```
NESN 4 409.2
AAPL 1905.0
<class '__main__.Position'> True
<class 'int'> <class 'str'> <class 'list'>
```

+ GOOD TO KNOW

What we leave out: object-oriented programming

You may see	What it is
<code>class Bond(Position):</code>	inheritance : a class built on top of another
<code>@dataclass</code> above a class	a shortcut that writes <code>__init__</code> for you
<code>def __repr__(self):</code>	a special method: how the object prints

Designing programs around classes and hierarchies is **object-oriented programming (OOP)**: a subject of its own, not part of this course.

```

1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total  < Moved into the loop
13
14 def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17 tickers = ["AAPL", "NESN", "V00"]
18 shares = [10, 4, 2]
19 price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21 total = total_value(tickers, shares, price)
22 print(f"total {total:,.2f}", round(total, 1))
23 print(years_to_double(), years_to_double(rate=0.07))
24 print(position_value(price=102.3, shares=4), sum(shares))
25 print(", ".join(tickers).lower(), type(total_value))

```

Spot the lie

A classmate indented line 12 by four more spaces. It runs, and every line prints.

Is the total right? Which kind of failure is this, and how would you catch it?

```

$ python portfolio.py
total 1,905.00 1905.0  < Is this right?
15 11
409.2 16
aapl, nesn, voo <class 'function'>

```

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total  < After the loop: once
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
```

```
total 3,338.20 3338.2  < The right total
```

```
15 11
```

```
409.2 16
```

```
aapl, nesn, voo <class 'function'>
```

The lie, explained

Indented, `return total` sits **inside the loop**. `return` ends the function **immediately**, so the loop stops after the first position: the "total" is AAPL alone, `1,905.00`.

Failure kind 3: it runs and lies. **Catch it with a rough check:** three positions cannot be worth exactly the first one. Or test the function on numbers you can add in your head.

Session 3: indentation is syntax. `return` makes it matter even more.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

Read it again

The same 25 lines as at the start. Every line is now yours:

- `import` on line 2, and `math.ceil` on line 15;
- three **definitions**, their **signatures**, a **docstring**, and `return`;
- **local** and **global** names, including the two `total`s;
- **positional**, **keyword** and **default** arguments on lines 21–24;
- a **method chain** on line 25.

portfolio.py

```
1  # portfolio.py – the ledger, organised into functions
2  import math
3
4  def position_value(shares, price):
5      """Value of one position, in CHF."""
6      return shares * price
7
8  def total_value(tickers, shares, price):
9      total = 0.0
10     for i in range(len(tickers)):
11         total += position_value(shares[i], price[tickers[i]])
12     return total
13
14  def years_to_double(rate=0.05):
15     return math.ceil(math.log(2) / math.log(1 + rate))
16
17  tickers = ["AAPL", "NESN", "V00"]
18  shares = [10, 4, 2]
19  price = {"AAPL": 190.5, "NESN": 102.3, "V00": 512.0}
20
21  total = total_value(tickers, shares, price)
22  print(f"total {total:,.2f}", round(total, 1))
23  print(years_to_double(), years_to_double(rate=0.07))
24  print(position_value(price=102.3, shares=4), sum(shares))
25  print(", ".join(tickers).lower(), type(total_value))
```

```
$ python portfolio.py
total 3,338.20 3338.2
15 11
409.2 16
aapl, nesn, voo <class 'function'>
```

Check yourself

On paper first, then check with your neighbour:

1. What does `position_value(shares=2, price=512.0)` return?
2. What would `years_to_double(0.07)` return? Is that call positional or keyword?
3. After line 21, what is the value of `i`? Of `total`?
4. What is the difference between `arr.shape` and `arr.sum()`?
5. `x = [3, 1, 2].sort()`: what is `x`?

What you can now do

- Read any call: function, arguments, and the value it is replaced by.
- Write a function with `def`, a docstring and `return`, and say why `print` is not `return`.
- Match arguments to parameters: by position, by keyword, and with defaults.
- Say which names are local and which are global.
- Tell a function (`sum(x)`) from a method (`x.sum()`), and an attribute (`arr.shape`) from a method.
- Import modules, and say what a class, an object and an instance are.
- Read and write a simple class: a constructor, attributes and methods.